

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: CSA51 **COURSE NAME:** Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO (BL3, BL6)*

Assessment Tool Weightage: 10%

QUESTIONS: 5

TOTAL MARKS: 50

Annexure A

Comparative Analysis

Code of Conduct:

I Gurusurya G 192311332_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Gurusurya
Signature of the student:

Annexure A
Comparative Analysis

1. Compare Authentication Header (AH) and Encapsulating Security Payload (ESP) in IPSec with respect to security services, performance overhead, and suitability for different network scenarios. Analyze which is more appropriate for a VPN deployment.
2. Compare transport mode and tunnel mode in IPSec architecture. Evaluate their effectiveness in securing host-to-host versus network-to-network communication.
3. Perform a comparative analysis of Pretty Good Privacy (PGP) and S/MIME in terms of key management, trust models, scalability, and enterprise usability. Recommend the most suitable approach for a corporate email system.
4. Compare rule-based and machine learning-based email spam detection techniques. Analyze their effectiveness in handling evolving spam patterns and minimizing false positives.
5. Compare manual key management and automated key management (e.g., using IKE) in IPSec. Evaluate their impact on security, scalability, and operational complexity in large organizations.

1. Authentication Header (AH) vs. Encapsulating Security Payload (ESP)

IPSec secures IP traffic at the network layer using two alternative protection headers, AH and ESP. Although both can be used to protect a communication session, they differ fundamentally in the security services they provide, the way they process a packet, and the performance cost they impose.

1.1 Security Services

AH (IP protocol 51) provides connectionless integrity, data-origin authentication, and optional anti-replay protection for the entire IP packet, including portions of the outer IP header that do not change in transit. It does this by computing a keyed hash (an Integrity Check Value, ICV) over the packet and inserting it as a new header. Because AH authenticates the IP header itself, it can detect any tampering with source and destination addresses, which is valuable for detecting spoofing. However, AH does not encrypt the payload at all, so it offers no confidentiality.

ESP (IP protocol 50) provides confidentiality through symmetric encryption of the payload, together with optional data-origin authentication, integrity, and anti-replay protection for the encapsulated data. Unlike AH, ESP's integrity check does not cover the outer IP header (in transport mode), because the header must remain mutable for routing. ESP is therefore the only one of the two protocols capable of hiding the content of communication from eavesdroppers, and in practice it is used far more widely because most deployments need confidentiality, not just authentication.

1.2 Performance Overhead

AH is computationally lighter than ESP because it only performs a hashing operation (e.g., HMAC-SHA-256) and does not perform encryption or decryption. This makes AH attractive in scenarios where processing power is constrained but authenticity must still be guaranteed. ESP carries a higher processing cost because it must encrypt and decrypt the payload in addition to computing the ICV, and it also adds padding to align data to block-cipher boundaries, which increases packet size slightly more than AH. On modern hardware with AES-NI or similar cryptographic acceleration, this difference is small, but on constrained embedded devices or very high-throughput links it can still be measurable.

1.3 Suitability for Different Network Scenarios

Criterion	AH	ESP
Confidentiality	Not provided	Provided (payload encryption)
Integrity / Authentication	Provided for entire packet including outer IP header	Provided for payload and ESP header only
NAT compatibility	Poor — NAT changes the IP header, breaking the ICV	Good, especially with NAT-Traversal (UDP encapsulation)
Processing overhead	Low (hashing only)	Moderate to high (encryption + optional hashing)
Typical use case	Environments needing origin authentication without secrecy, e.g., internal routing integrity	VPNs, remote access, confidential data transport across untrusted networks
IP protocol number	51	50

AH's inability to traverse Network Address Translation (NAT) devices is a significant practical limitation, since it authenticates the outer IP header and any address rewriting invalidates the ICV. ESP, particularly when combined with NAT-Traversal (NAT-T) encapsulation in UDP, works reliably across NAT boundaries, which are extremely common on the modern Internet. This alone makes ESP the dominant choice in almost

all commercial and open-source VPN products.

1.4 Recommendation for VPN Deployment

Recommendation

For virtual private network deployments, ESP is the more appropriate protocol. VPNs typically need to protect sensitive data traversing an untrusted public network, which requires confidentiality that only ESP can provide. ESP also supports integrity and authentication when configured with an authenticated encryption suite (e.g., AES-GCM) or combined with HMAC, so it can deliver every security service AH offers, plus encryption, while also traversing NAT gateways correctly. AH remains useful only in narrow scenarios where encryption is legally restricted or unnecessary but strict header-level authentication is required; in virtually all mainstream VPN products (IKEv2/IPSec, L2TP/IPSec, most site-to-site VPNs) ESP alone, or ESP with authentication enabled, is used and AH is rarely deployed today.

2. Transport Mode vs. Tunnel Mode in IPSec

IPSec can operate in one of two modes that determine how much of the original IP packet is protected and how the resulting packet is addressed. The choice of mode is independent of whether AH or ESP is used, but it strongly affects which network scenario the configuration is suited for.

2.1 Transport Mode

In transport mode, IPSec protects only the payload of the original IP packet (e.g., the TCP/UDP segment); the original IP header is retained and used for routing, with the AH or ESP header inserted between the IP header and the transport-layer payload. Because the original source and destination addresses remain visible, transport mode is inherently a host-to-host mechanism: it secures the communication between the actual endpoints that are exchanging data, and it does not conceal the identity of those endpoints from anyone observing the traffic. Its overhead is lower because no new outer IP header is added, and it is well suited to end-to-end protection between two hosts that are themselves the source and destination of the traffic, such as securing traffic between a client machine and a specific server.

2.2 Tunnel Mode

In tunnel mode, the entire original IP packet — header and payload — is encapsulated inside a new IP packet with a new outer header. The AH or ESP protection is applied to the original packet as a whole, and the new outer header carries the addresses of the IPSec gateways (for example, two VPN routers or firewalls) rather than the original hosts. Tunnel mode therefore hides the internal, original addressing information from external observers and allows two private networks, each behind its own security gateway, to communicate securely across an untrusted network such as the Internet without exposing their internal topology. It carries a higher overhead than transport mode because of the additional outer IP header, but this overhead is generally a worthwhile trade-off for the additional confidentiality and topology-hiding it provides.

2.3 Comparative Summary

Aspect	Transport Mode	Tunnel Mode
What is protected	Payload of the original packet only	Entire original IP packet (header + payload)
Header handling	Original IP header preserved and reused	New outer IP header added; original header encapsulated
Addressing exposed	Original host addresses visible	Only gateway addresses visible externally
Overhead	Lower (no extra IP header)	Higher (extra IP header and encapsulation)
Best-fit topology	Host-to-host, end-to-end	Network-to-network, gateway-to-gateway, remote access
Typical deployment	Securing a specific application session between two endpoints	Site-to-site VPNs, remote-access VPNs

2.4 Effectiveness for Host-to-Host vs. Network-to-Network Communication

For host-to-host communication, transport mode is more effective and efficient. Since both communicating parties are IPSec-aware endpoints, there is no need to hide their addresses from each other, and avoiding the extra encapsulation reduces bandwidth consumption and processing latency — an important consideration for high-volume, low-latency traffic such as database replication or real-time application traffic between two known servers.

For network-to-network communication, tunnel mode is clearly superior. When two entire private networks need to communicate through security gateways rather than through IPSec-capable end hosts, tunnel mode allows the gateways to perform encryption and decryption on behalf of all internal hosts, none of which need to run IPSec themselves. It also protects internal addressing schemes, which may use private IP ranges that are not routable on the public Internet, by wrapping them inside a routable outer header. This is precisely the pattern used in typical site-to-site VPNs connecting branch offices to a corporate headquarters, and it is also the pattern used by most remote-access VPN clients, which tunnel an internal, dynamically assigned address inside a packet addressed to the corporate gateway.

In short, transport mode should be chosen when the communicating parties are the IPSec endpoints themselves and minimal overhead is desired, while tunnel mode should be chosen whenever traffic must cross intermediary security gateways or when the internal network topology needs to remain hidden from the public network.

3. PGP vs. S/MIME for Secure Email

Pretty Good Privacy (PGP) and Secure/Multipurpose Internet Mail Extensions (S/MIME) are the two dominant standards for providing confidentiality, integrity, authentication, and non-repudiation for electronic mail. Both use public-key cryptography combined with symmetric encryption for the message body, but they differ sharply in how they establish trust and how well they scale within organizations.

3.1 Key Management

PGP uses a decentralized key-management model. Each user generates their own key pair and distributes the public key directly to correspondents or publishes it on a public key server; there is no mandatory central authority responsible for issuing or vouching for keys. Users are themselves responsible for safeguarding private keys, generating revocation certificates, and refreshing keys as needed.

S/MIME relies on a centralized, hierarchical Public Key Infrastructure (PKI). Digital certificates are issued by a Certificate Authority (CA) that verifies the identity of the requester before binding a public key to that identity in an X.509 certificate. Key issuance, renewal, and revocation are handled through this formal CA hierarchy, typically integrated with an organization's existing PKI or a commercial CA.

3.2 Trust Models

PGP implements the Web of Trust model, in which users sign each other's public keys to vouch for their authenticity, and trust is established transitively through chains of signatures among individuals who know and can vouch for one another. This model is flexible and does not depend on any single organization, but it can be difficult to bootstrap in a large enterprise because trust paths between arbitrary employees may not exist.

S/MIME uses a hierarchical trust model rooted in one or more trusted CAs. Any certificate issued by a CA that a mail client trusts is automatically trusted, without individual users needing to vouch for one another. This mirrors the trust model already used for HTTPS/TLS on the web, making it conceptually familiar and straightforward to automate in corporate identity systems.

3.3 Scalability and Enterprise Usability

Criterion	PGP	S/MIME
Key issuance model	Decentralized, self-generated key pairs	Centralized, CA-issued X.509 certificates
Trust model	Web of Trust (peer signatures)	Hierarchical PKI (CA-rooted)
Ease of onboarding new users	Manual key exchange/verification needed	Automated via existing corporate PKI/Active Directory
Scalability in large organizations	Weaker — trust paths do not scale well with headcount	Strong — CA hierarchy scales naturally with directory services
Native client support	Requires plugins/third-party tools in many clients	Built into most enterprise mail clients (Outlook, Apple Mail)
Regulatory/audit alignment	Less standardized for compliance reporting	Aligns well with existing corporate PKI and compliance tooling
Typical user base	Individuals, activists, open-source communities, journalists	Corporations, government agencies, regulated industries

3.4 Recommendation for a Corporate Email System

Recommendation

S/MIME is the more suitable approach for a corporate email system. Its hierarchical PKI model integrates cleanly with directory services such as Active Directory, allows automated certificate

issuance and revocation, is natively supported by mainstream enterprise mail clients, and scales predictably as the organization grows, all of which are difficult to achieve with PGP's peer-to-peer Web of Trust in a large workforce. PGP remains preferable for smaller, decentralized, or cross-organizational communities where no shared CA exists and users are willing to manage their own keys, but for enterprise deployment S/MIME is generally the pragmatic and industry-standard choice.

4. Rule-Based vs. Machine Learning-Based Spam Detection

Email spam filtering has evolved from static, human-authored rules to adaptive statistical and machine-learning models. Both approaches remain in use today, frequently in combination, but they differ substantially in how they identify spam and how well they adapt over time.

4.1 Rule-Based Detection

Rule-based filters classify messages using a predefined set of conditions, such as blacklisted sender domains, banned keywords or phrases, suspicious header patterns, DNS-based blackhole list lookups, or SPF/DKIM/DMARC authentication failures. Each rule is manually authored by a security analyst and typically assigned a weight; a message is flagged as spam once its cumulative rule score crosses a threshold. This approach is transparent, deterministic, and fast to execute, and it is easy for an administrator to understand exactly why a particular message was blocked.

However, rule-based systems are inherently reactive: a rule can only detect a pattern that a human has already observed and encoded. Spammers routinely defeat static rules through simple obfuscation techniques, such as misspelling trigger words, embedding text inside images, or using freshly registered domains that have not yet been blacklisted, which forces administrators into a continuous, labor-intensive cycle of rule updates.

4.2 Machine Learning-Based Detection

Machine learning-based filters, most commonly based on Naive Bayes classifiers, support vector machines, random forests, or increasingly deep neural networks, learn statistical patterns from large labeled datasets of spam and legitimate email. Rather than relying on a fixed set of hand-written conditions, these models learn which combinations of features, such as word frequencies, sender reputation signals, message structure, and behavioral metadata, are statistically associated with spam, and they can be retrained continuously as new data arrives. This allows machine-learning filters to adapt automatically to new spam campaigns without requiring a human to manually write a new rule for every variant.

The trade-off is reduced transparency (many ML models, particularly deep learning ones, function as black boxes that are hard to interpret), a dependency on the quality and volume of training data, susceptibility to adversarial manipulation designed specifically to evade the model, and the computational cost of training and periodically retraining the model.

4.3 Comparative Analysis

Criterion	Rule-Based	Machine Learning-Based
Adaptability to new/evolving spam	Low — requires manual rule updates	High — retrains on new patterns automatically
Transparency / explainability	High — each rule is human-readable	Lower — especially for complex models
Initial setup effort	Moderate — write and tune rules	High — requires labeled training data and model tuning
Ongoing maintenance	Continuous manual rule updates	Periodic retraining, largely automatable
False positive control	Predictable but brittle at edge cases	Can be tuned statistically, but errors are harder to trace
Resistance to obfuscation	Weak — easily evaded with minor changes	Stronger — recognizes underlying statistical patterns
Resource requirements	Low computational cost	Higher (training/inference compute)

In practice, minimizing false positives is easier with rule-based systems for well-understood, unambiguous

cases (for example, a hard block on a domain already confirmed as malicious), because the logic is explicit and predictable. Machine-learning systems, given sufficient and representative training data, generally achieve better overall precision and recall across the full spam landscape because they can weigh many weak signals simultaneously rather than relying on a small number of hard-coded triggers, but they require ongoing monitoring to catch model drift, where classification accuracy degrades as spam tactics evolve faster than the training data reflects.

Most modern commercial email security platforms use a hybrid architecture: fast rule-based filters and authentication checks (SPF/DKIM/DMARC, blacklists) handle the clearest, lowest-risk cases at minimal computational cost, while machine-learning models handle the harder, evolving cases and continuously adapt to new campaigns. This combination captures the interpretability and speed of rules together with the adaptability of statistical learning.

5. Manual vs. Automated (IKE-Based) Key Management in IPSec

Every IPSec Security Association (SA) requires shared cryptographic keys for encryption and/or authentication. IPSec supports two approaches to establishing and maintaining these keys: manual key management, where keys are configured directly by an administrator, and automated key management, typically performed by the Internet Key Exchange (IKE) protocol.

5.1 Manual Key Management

With manual key management, an administrator generates the shared keys and manually configures them, identically, on both endpoints of the IPSec connection. No key-negotiation protocol runs; the Security Association parameters and keys are simply entered as static configuration. This approach is conceptually simple and avoids the overhead of a negotiation protocol, but it has serious limitations: keys do not change automatically, so a compromised key remains valid indefinitely until an administrator notices and manually replaces it on every device; distributing keys securely to many devices is itself a hard problem; and the process does not scale, since every new tunnel or key rotation requires manual, synchronized configuration changes across all affected endpoints.

5.2 Automated Key Management with IKE

Automated key management uses IKE (IKEv1 or the more modern IKEv2) to dynamically negotiate, authenticate, and periodically refresh Security Associations and their keys, typically using the Diffie-Hellman algorithm to derive shared secrets without transmitting the key material itself over the network. IKE also handles peer authentication (via pre-shared keys, digital certificates, or EAP methods), negotiates cryptographic algorithms, and automatically re-keys sessions at configurable intervals, which limits the amount of data protected by any single key and significantly reduces the impact of a potential key compromise. Because the entire process is automated, adding new tunnels, rotating keys, or renegotiating parameters requires no manual synchronization between endpoints beyond the initial trust configuration (such as a shared PSK or a certificate hierarchy).

5.3 Comparative Analysis

Criterion	Manual Key Management	Automated (IKE-Based)
Key rotation	Manual, infrequent, error-prone	Automatic, on a configurable schedule
Security exposure from compromise	High — compromised key persists until manually replaced	Low — periodic re-keying limits exposure window
Peer authentication	Implicit, based on matching pre-shared keys	Explicit, via certificates, PSKs, or EAP within IKE
Scalability	Poor — linear administrative effort per tunnel	Strong — scales to hundreds/thousands of tunnels
Operational complexity	Low initial complexity, high ongoing burden	Higher initial setup, low ongoing burden
Suitability for dynamic environments	Poor (e.g., mobile/remote-access users)	Excellent — supports dynamic peers and NAT-T
Typical use case	Small, static, point-to-point test or lab links	Enterprise VPNs, cloud interconnects, remote access at scale

5.4 Impact on Large Organizations

For a large organization, automated key management is essentially a prerequisite, not merely an enhancement. Manually managing distinct keys across dozens or hundreds of VPN tunnels connecting branch offices, cloud environments, and remote workers would require an administrative team to

coordinate simultaneous, error-free key updates across every affected device whenever a rotation is needed, a process that does not scale and is highly prone to human error and operational downtime. Automated key management via IKE removes this burden by handling negotiation, authentication, and periodic re-keying without administrator intervention beyond the initial trust setup, and it integrates naturally with certificate-based PKI, which itself scales well through automated issuance and revocation.

The security benefit compounds the operational one: because IKE regularly derives fresh session keys through Diffie-Hellman exchange (optionally with Perfect Forward Secrecy), the amount of traffic exposed by any single compromised key is bounded, whereas a manually configured key that is rarely rotated in practice represents a long-lived single point of failure. For these reasons, virtually all enterprise-grade IPSec deployments use IKEv2 for automated key management, reserving manual keying for small-scale testing, legacy systems, or highly specialized environments with only a handful of static, long-lived tunnels.

5.5 Recommendation

Recommendation

Large organizations should use automated key management via IKEv2 as the default for all production IPSec deployments. It scales to large numbers of tunnels, supports dynamic and mobile endpoints, integrates with certificate-based authentication, and substantially reduces the security risk associated with long-lived static keys. Manual key management should be reserved for small, controlled scenarios such as lab testing or a handful of permanent point-to-point links where the operational overhead is genuinely manageable.

Conclusion

Across all five comparisons, a common theme emerges: the more flexible, higher-overhead option is generally preferred whenever the deployment must scale across an untrusted network or a large, dynamic organization, while the leaner, simpler option remains valuable only in narrow, constrained, or small-scale scenarios. ESP outperforms AH for VPNs because confidentiality and NAT compatibility matter more than AH's marginally lower processing cost. Tunnel mode outperforms transport mode for network-to-network links because it hides topology and protects entire packets across gateways, while transport mode remains the efficient choice for direct host-to-host sessions. S/MIME outperforms PGP in the enterprise because centralized PKI scales with organizational growth in a way that the Web of Trust cannot. Machine-learning spam detection outperforms static rules against evolving threats, though the strongest real-world systems combine both. Finally, automated IKE-based key management outperforms manual keying in any organization beyond a handful of static links, both operationally and from a security-exposure standpoint. In each case, the recommended choice reflects the specific security services, performance costs, and operational scale the pair of technologies was designed to address.